

04 - Flyway Database Migration

How the database structure is created, versioned and changed safely.

What is this?

Flyway is a notebook that records every change ever made to the database.

Each change is one numbered file. Flyway keeps a table listing which files it has already run. On every start-up it compares the two and runs only what is new.

Like: the maintenance logbook for the shop's almirah. "Change 1: built it with four shelves. Change 2: added a lock to the top shelf." Hand the logbook to a carpenter and they can rebuild it exactly.

WHY

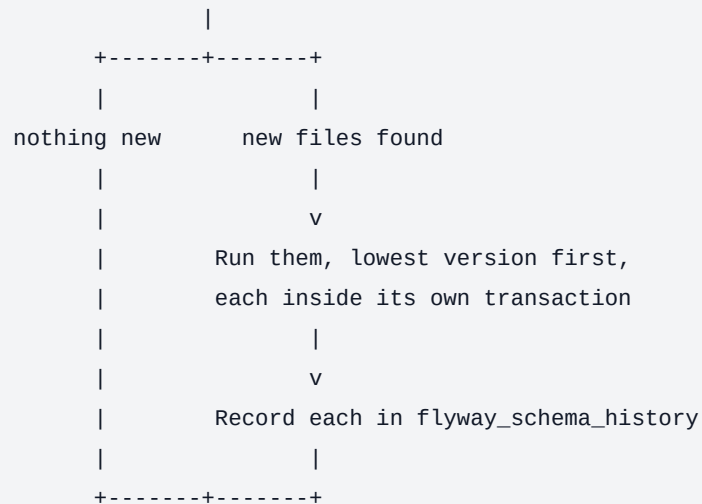
Why not just create the tables by hand

Because you would have to do it identically on your laptop, on your colleague's laptop, on the test machine and on the shop's server - and remember every change you made along the way. One forgotten column and the application crashes in a way that takes hours to trace. Flyway makes the structure reproducible from a file that lives in git.

What happens when the application starts

```
Application starts
  |
  v
Spring Boot opens a connection to PostgreSQL
  |
  v
FLYWAY STARTS
  |
  v
Does the table flyway_schema_history exist?
  |           |
no          yes
  |           |
create it    |
  |           |
+-----+-----+
      v
Read every file in db/migration (and db/seed in dev)
  |
  v
```

Compare against what history says has already run



Hibernate validates: do the Java entities match these tables?

Application finishes starting

IMPORTANT

If a migration fails, the application does not start

This is deliberate. A half-created database is far more dangerous than an application that refuses to run, because the application would then write bad data into a structure it misunderstands.

The naming rule

```
V1__auth_schema.sql
^ ^ ^
| | +--- description, underscores become spaces
| +----- TWO underscores. Not one.
+----- V for versioned, then the version number
```

IMPORTANT

Two underscores

V1_auth_schema.sql with one underscore is silently ignored. This catches everybody once.

This project's actual migrations

There are exactly two. Do not invent others.

File	Location	Runs in
V1__auth_schema.sql	backend/src/main/resources/db/migration/	every environment

File	Location	Runs in
V900__seed_dev_data.sql	backend/src/main/resources/db/seed/	development and test only

V1 - the schema

Creates seven tables, plus one helper function:

Table	Holds
permission	The catalogue of things that can be permitted - 31 rows seeded
role	Named permission bundles - OWNER, MANAGER, ACCOUNTANT, STAFF
role_permission	Which role has which permission
app_user	The people who sign in
refresh_token	One row per active session
password_reset_token	Single-use reset links
security_audit_log	Sign-ins, password changes, token misuse

It also creates `set_updated_at()`, a trigger function that keeps the `updated_at` column current on the `permission` table.

WHY

Why `app_user` and not `user`

`USER` is a reserved word in PostgreSQL - it is a built-in function that returns the current database user. A table called `user` would need quoting in every single query. Naming it `app_user` avoids the problem permanently.

V900 - the development seed data

Twelve users, one custom role and twelve audit records, so that every screen has something to show and pagination, search and filtering can be tested.

IMPORTANT

Why the version number is 900

The seed file lives in a **different folder** - `db/seed` - which is listed in the Flyway locations for the dev and test profiles only:

```
# application-dev.yml
locations: classpath:db/migration,classpath:db/seed

# application-prod.yml
locations: classpath:db/migration
```

Production **physically cannot load it**. Without this split, twelve accounts with publicly known passwords would be created on the shop's live system on the first start-up. The high number keeps it last if it ever does run.

The history table

Flyway keeps its own record. Look at it any time:

RUN THIS

See what has run

```
docker exec -it hardware-erp-postgres psql -U hardware_erp -d hardware_erp \  
-c "SELECT version, description, success, installed_on FROM flyway_schema_history ORDER  
BY installed_rank;"
```

WHAT YOU SHOULD SEE

Expected on a dev machine

version	description	success	installed_on
1	auth schema	t	2026-08-14 09:12:03.114
900	seed dev data	t	2026-08-14 09:12:03.402

success = t means it completed. On a production machine you should see only version 1.

Flyway also stores a **checksum** of each file. If you edit a migration that has already run, the checksum no longer matches and Flyway refuses to start.

IMPORTANT

Never edit a migration that has already run

This is the single most important rule. Once `V1__auth_schema.sql` has run anywhere - even on your own laptop - it is frozen. To change the structure, add `V2__whatever.sql`.

The reason: your colleague's database already ran the old V1. Editing the file does not change their database, so their structure and yours silently diverge.

Adding a new migration

Say Module 2 needs a supplier table.

1. Create `backend/src/main/resources/db/migration/V2__supplier_schema.sql`
2. Write the `CREATE TABLE` statement using this project's conventions:

```
CREATE TABLE supplier (  
    supplier_id BIGINT GENERATED BY DEFAULT AS IDENTITY,  
    supplier_name VARCHAR(255) NOT NULL,  
    mobile_no VARCHAR(15) NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',  
    created_at TIMESTAMP(3) NOT NULL,  
    created_by BIGINT,  
    updated_at TIMESTAMP(3),  
    updated_by BIGINT,
```

```
version          INTEGER          NOT NULL DEFAULT 0,
CONSTRAINT pk_supplier PRIMARY KEY (supplier_id),
CONSTRAINT uk_supplier_mobile UNIQUE (mobile_no),
CONSTRAINT ck_supplier_status CHECK (status IN ('ACTIVE', 'INACTIVE'))
);
```

3. Restart the backend. Flyway notices V2 and runs it.

DEPENDENCY

The conventions this project locks

- BIGINT GENERATED BY DEFAULT AS IDENTITY for primary keys, not SERIAL
- TIMESTAMP(3) for date-and-time, never DATETIME
- VARCHAR(20) plus a CHECK for status columns, never a PostgreSQL ENUM type
- Every constraint explicitly named, so an error message identifies the rule
- The four audit columns on every table

These are recorded in `project-knowledge/DATABASE_REGISTRY.md` and enforced by `registry/static_check.py`.

Changing an existing table safely

You want to add a column to `app_user`. Do **not** edit V1.

RUN THIS

Create V3__add_user_photo.sql

```
ALTER TABLE app_user ADD COLUMN photo_path VARCHAR(500);
```

Adding a nullable column is safe: existing rows get `NULL`.

Adding a `NOT NULL` column to a table that already has rows is not, because

those rows have no value to put there. Do it in three steps:

```
ALTER TABLE app_user ADD COLUMN branch_code VARCHAR(10);
UPDATE app_user SET branch_code = 'MAIN' WHERE branch_code IS NULL;
ALTER TABLE app_user ALTER COLUMN branch_code SET NOT NULL;
```

IMPORTANT

Never change the production schema by hand

If you run an `ALTER TABLE` directly in pgAdmin on the shop's server, Flyway does not know about it. The next deployment will either fail validation or apply a change on top of a structure it does not recognise. Every structural change goes through a migration file, without exception.

Hibernate's role

After Flyway finishes, Hibernate checks the tables match the Java entity classes:

```
spring.jpa.hibernate.ddl-auto: validate
```

Value	What it does	Used here?
validate	Check and complain. Change nothing	Yes
update	Try to alter tables to match the code	Never
create-drop	Delete everything and rebuild on every start	No

IMPORTANT

Why never "update"

update looks convenient and is a trap. It never drops or renames anything, so your database slowly fills with orphaned columns nobody can account for, and the structure ends up different on every machine. Flyway owns the schema. Hibernate only checks it.

If it fails

IF IT FAILS

"Validate failed: Migration checksum mismatch for version 1"

Somebody edited a migration that had already run. Either restore the original file exactly, or - **in development only** - wipe and start again:

```
docker compose down -v && docker compose up -d
```

IF IT FAILS

"Migration V1__auth_schema.sql failed"

The console prints the failing SQL line and the PostgreSQL error. Read the error itself, fix the SQL, then clean up the failed record before retrying:

```
DELETE FROM flyway_schema_history WHERE success = false;
```

IF IT FAILS

"Schema-validation: missing table [app_user]"

Flyway did not run, so Hibernate found nothing to validate. Check `spring.flyway.enabled` is true, and that the migration files are really in `src/main/resources/db/migration`.

IF IT FAILS

The seed users are missing

You are not running the dev profile. The console line

The following 1 profile is active: "dev" confirms it. Production omits `db/seed` on purpose.

Verify

VERIFY

Flyway is working correctly when

1. flyway_schema_history shows version 1 with success = t
2. \dt lists seven tables plus the history table
3. On a dev machine, version 900 is also present and app_user has 12 rows
4. Restarting the backend runs no migrations and logs
Schema is up to date. No migration necessary